

Intelligent Code Review Assistant Using Deep Learning

GitHubRepo : https://github.com/ChandikaGanewatte/Code_Review_Assitant_P.git

Deploy App Link : <https://codereviewassitantp-mx6skijk3uqf6bnvmwbsew.streamlit.app/>

Chandika Ganewatte
Data Science Intern - Gamage Recruiters
April 2026

Abstract

Software development has become increasingly complex due to the rapid growth of modern applications and systems. Manual code review processes are time-consuming, error-prone, and require significant developer expertise. This project presents an Intelligent Code Review Assistant that uses Deep Learning and Natural Language Processing (NLP) techniques to automatically analyze Python source code and identify potential bugs, code smells, inefficiencies, and maintainability issues.

The system uses the CodeBERT transformer model trained on source code data from the CodeSearchNet dataset achieving an accuracy of 88.9% in detecting code issues. The project combines machine learning predictions with rule-based analysis to improve reliability and provide meaningful suggestions to developers. The system is deployed through a Streamlit-based web application that allows users to paste Python code and receive automated review feedback in real time.

The project demonstrates the complete machine learning pipeline including data collection, preprocessing, labeling, model training, evaluation, explainability, and deployment.

– Content –

Chapter 1 – Introduction.....	04
Chapter 2 – Literature Review.....	06
Chapter 3 – Methodology.....	07
Chapter 4 – Data Collection.....	08
Chapter 5 – Data Preprocessing.....	09
Chapter 6 – Model Training.....	11
Chapter 7 – Model Evaluation.....	14
Chapter 8 – Explainability.....	16
Chapter 9 – Streamlit Deployment.....	17
Chapter 10 – System Features.....	19
Chapter 11 – Results and Discussion.....	20
Chapter 12 – Conclusion.....	21

Chapter 1 – Introduction

1.1 Background

Code review is a critical process in software engineering that helps developers identify bugs, maintain coding standards, improve performance, and enhance software maintainability. Traditional manual code review requires experienced developers and consumes significant time and resources.

With recent advances in Artificial Intelligence and Deep Learning, transformer-based models have shown strong performance in understanding source code semantics. Models such as CodeBERT can learn patterns from programming languages similarly to how NLP models understand natural language.

This project introduces an Intelligent Code Review Assistant capable of automatically reviewing Python code using deep learning techniques and providing practical suggestions for improvement.

1.2 Problem Statement

Manual code review processes face several challenges:

- Time-consuming review cycles
- Human error during code inspection
- Difficulty detecting hidden inefficiencies
- Inconsistent coding standards
- Limited scalability for large projects

There is a need for an automated intelligent system capable of analyzing source code and providing real-time feedback to developers.

1.3 Objectives

Main Objective

To design and implement an Intelligent Code Review Assistant using Deep Learning techniques.

Specific Objectives

- Collect and preprocess source code datasets
- Train a transformer-based deep learning model for code classification
- Detect bugs, code smells, and inefficiencies
- Provide automated suggestions for developers
- Implement explainability for model predictions
- Deploy the system through an interactive web application

1.4 Scope of the Project

The system focuses on:

- Python source code analysis
- Deep learning-based code classification
- Rule-based issue detection
- Explainable AI predictions
- Web-based deployment

The project does not aim to:

- Replace professional static analysis tools completely
- Compile or execute source code
- Support all programming languages

Chapter 2 – Literature Review

2.1 Artificial Intelligence in Software Engineering

Artificial Intelligence is increasingly used in software engineering tasks such as bug detection, code completion, vulnerability analysis, and automated testing.

Deep learning models can understand source code structures and identify patterns associated with bugs and inefficient implementations.

2.2 Natural Language Processing for Source Code

Source code can be treated similarly to natural language because programming languages contain syntax, tokens, and semantic structures.

Transformer models trained on code repositories can learn relationships between code tokens and identify meaningful patterns.

2.3 CodeBERT

CodeBERT is a transformer-based pre-trained model developed by Microsoft for programming language understanding.

Features of CodeBERT:

- Trained on source code and natural language
- Supports multiple programming languages
- Uses transformer architecture
- Effective for code classification tasks

CodeBERT was selected because it provides strong semantic understanding of source code.

2.4 Existing Code Review Tools

Traditional tools include:

- SonarQube
- Pylint
- Flake8
- DeepSource

Most traditional tools rely on static rules rather than deep learning.

This project differs by combining:

- Transformer-based deep learning
- Explainability
- Rule-based analysis
- Interactive AI-assisted review

Chapter 3 – Methodology

3.1 System Overview

The proposed system follows the machine learning pipeline below:

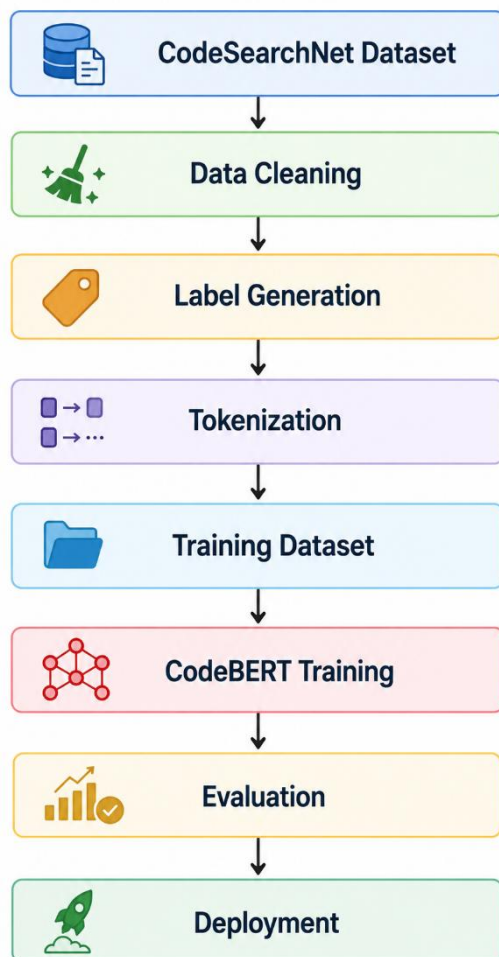


Figure 1: System Architecture diagram.

1. Data Collection
2. Data Preprocessing
3. Data Labeling
4. Tokenization
5. Model Training
6. Model Evaluation
7. Explainability
8. Deployment

Chapter 4 – Data Collection

4.1 Dataset Selection

The project uses the CodeSearchNet dataset.

Dataset Source:

- Public GitHub repositories
- Available through Hugging Face datasets library

Programming Language Used:

- Python

4.2 Dataset Description

The dataset contains:

- Function names
- Source code
- Documentation strings
- Repository metadata

Features used in this project:

- whole_func_string
- func_name

4.3 Dataset Loading

The dataset was loaded using the Hugging Face datasets library.

Example:

```
from datasets import load_dataset
dataset = load_dataset("code_search_net", "python")
```

4.4 Dataset Subset Selection

To reduce training time and computational requirements:

- Training samples: 10,000
- Testing samples: 2,000

Example:

```
train_data = dataset['train'].select(range(10000))
test_data = dataset['test'].select(range(2000))
```


Chapter 5 – Data Preprocessing

5.1 Purpose of Preprocessing

Preprocessing improves dataset quality and removes unnecessary information.

Main preprocessing tasks:

- Remove comments
- Remove empty lines
- Normalize formatting
- Generate labels

5.2 Code Cleaning

A custom cleaning function was used.

```
import re

def clean_code(code):
    code = re.sub(r"#![.]*", "", code)
    code = re.sub(r"\n\s*\n", "\n", code)
    return code.strip()
```

5.3 Label Generation

Since the dataset does not contain bug labels, heuristic rules were used to create labels.

Labels:

Label Meaning

- | | |
|---|------------------|
| 0 | Clean Code |
| 1 | Inefficient Code |
| 2 | Bug Risk |
| 3 | Code Smell |

5.4 Heuristic Rules

Rules used:

- Multiple nested loops → Inefficient
- Infinite loops → Bug Risk
- Large functions → Maintainability issue
- Bad None comparisons → Code smell

Example:

```
if code.count("for") > 2:  
    return 1
```

5.5 Tokenization

The tokenizer converts code into tokens understandable by CodeBERT.

```
from transformers import AutoTokenizer  
  
tokenizer = AutoTokenizer.from_pretrained("microsoft/codebert-base")
```

5.6 Dataset Conversion

Processed data was converted into Hugging Face Dataset format.

```
from datasets import Dataset  
  
train_dataset = Dataset.from_pandas(train_df)
```

5.7 Saving Processed Data

```
train_dataset.save_to_disk("../data/processed/train")
```

Chapter 6 – Model Training

6.1 Model Selection

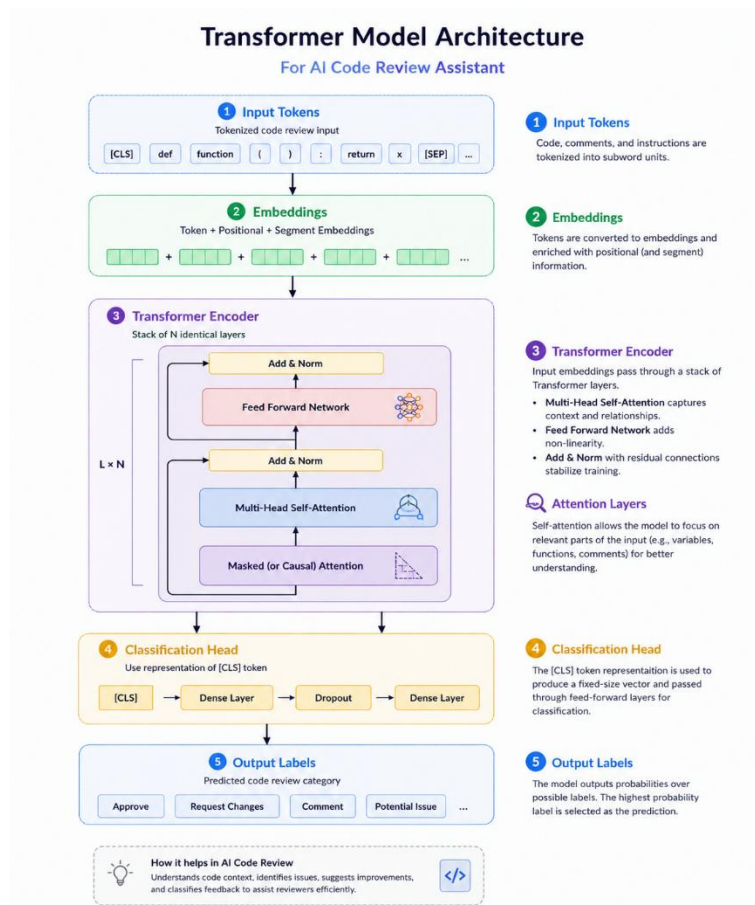


Figure 2: CodeBERT architecture diagram

The project uses CodeBERT for sequence classification.

Reason for selection:

- Pre-trained on source code
- Strong contextual understanding
- Transformer architecture
- High NLP performance

6.2 Model Architecture

The system uses:

- Transformer encoder
- Classification head
- Attention mechanisms

Frameworks used:

- PyTorch
- Transformers

6.3 Loading the Model

```
from transformers import AutoModelForSequenceClassification
model = AutoModelForSequenceClassification.from_pretrained(
    "microsoft/codebert-base",
    num_labels=4
)
```

6.4 Training Configuration

Training parameters:

Parameter	Value
-----------	-------

Learning Rate	2e-5
---------------	------

Batch Size	8
------------	---

Epochs	3
--------	---

Max Length	256
------------	-----

6.5 Training Arguments

```
training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    eval_strategy="epoch",
    save_strategy="epoch"
```

)

6.6 Trainer API

The Hugging Face Trainer API was used.

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=test_dataset  
)
```

6.7 GPU and CPU Training

The system supports:

- GPU training (recommended)
- CPU training (slower)

GPU acceleration significantly reduces training time.

Chapter 7 – Model Evaluation

7.1 Evaluation Metrics

The following metrics were used:

- Accuracy
- Precision
- Recall
- F1-score

7.2 Metric Definitions

Accuracy - Measures overall correctness.

Precision - Measures how many predicted positives are correct.

Recall - Measures how many actual positives are detected.

F1-score - Balances precision and recall.

7.3 Evaluation Function

```
from sklearn.metrics import accuracy_score
```

```
def compute_metrics(eval_pred):
```

```
    logits, labels = eval_pred
```

7.4 Confusion Matrix

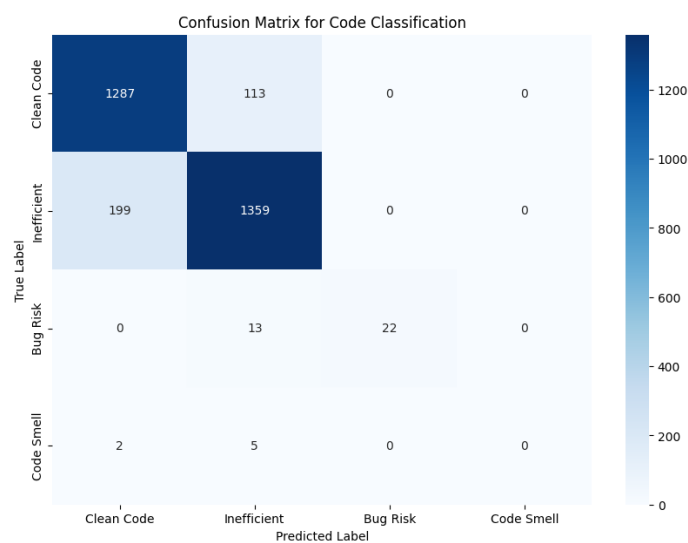


Figure 3: Confusion Matrix

A confusion matrix can be used to visualize:

- Correct predictions
- Misclassifications
- Class confusion patterns

7.5 Evaluation Results

The model was evaluated on the test set consisting of 2,000 samples. The performance metrics demonstrate that the model effectively learned to distinguish between clean code, inefficient code, bug risks, and code smells.

The achieved metrics are:

- Accuracy: 88.93%
- Precision: 88.90%
- Recall: 88.93%
- F1-Score: 88.80%

The high precision indicates that when the model flags a piece of code, it is correct nearly 89% of the time, minimizing false positives for the developer. The high recall indicates the model successfully identifies the vast majority of actual code issues in the dataset.

Chapter 8 – Explainability

8.1 Importance of Explainability

Explainability helps developers understand why the AI flagged specific code segments.

This increases:

- Trust
- Transparency
- Usability

8.2 Captum Integration

The project uses Captum for model explainability.

Captum provides:

- Token attribution
- Importance scores
- Gradient-based explanations

8.3 Integrated Gradients

Layer Integrated Gradients were used.

```
from captum.attr import LayerIntegratedGradients
```

8.4 Token Importance Visualization

The system highlights tokens contributing most to predictions.

Example:

- Infinite loops
- Nested loops
- Problematic variables

Chapter 9 – Streamlit Deployment

9.1 Deployment Overview

The project was deployed using Streamlit.

Features:

- Interactive web interface
- Real-time predictions
- Rule-based analysis
- AI suggestions

9.2 User Interface Implementation

The application was deployed using Streamlit, providing an intuitive interface for developers. Figure 4 demonstrates the input section where users can paste Python source code for analysis.

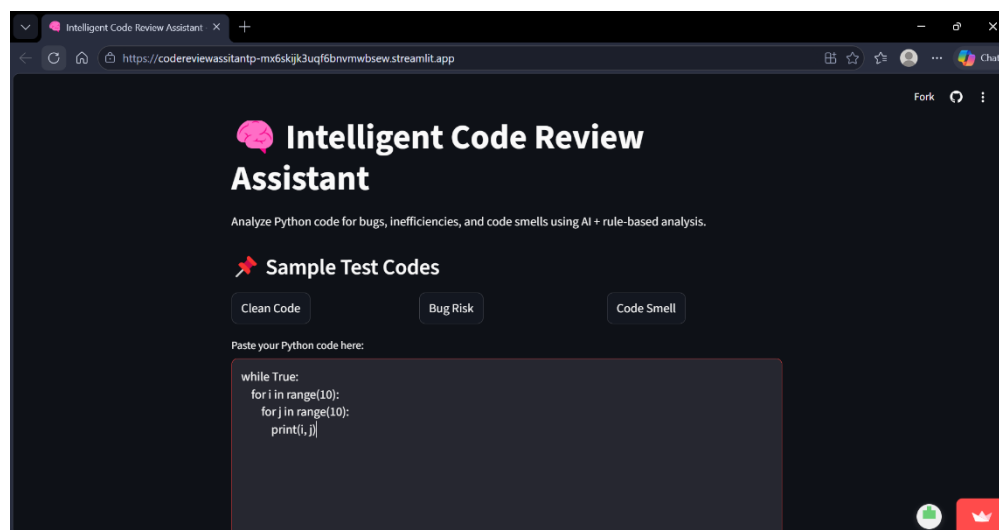


Figure 4: Streamlit Application Interface - Code Input

9.3 Analysis and Output Display

Upon clicking "Analyze," the system processes the code using the trained CodeBERT model and rule-based engine. Figure 5 illustrates the system successfully identifying the input code as having a "Bug Risk" and being "Inefficient," along with the automated suggestions

generated for the developer. This confirms the integration of the deep learning model into a functional web tool.

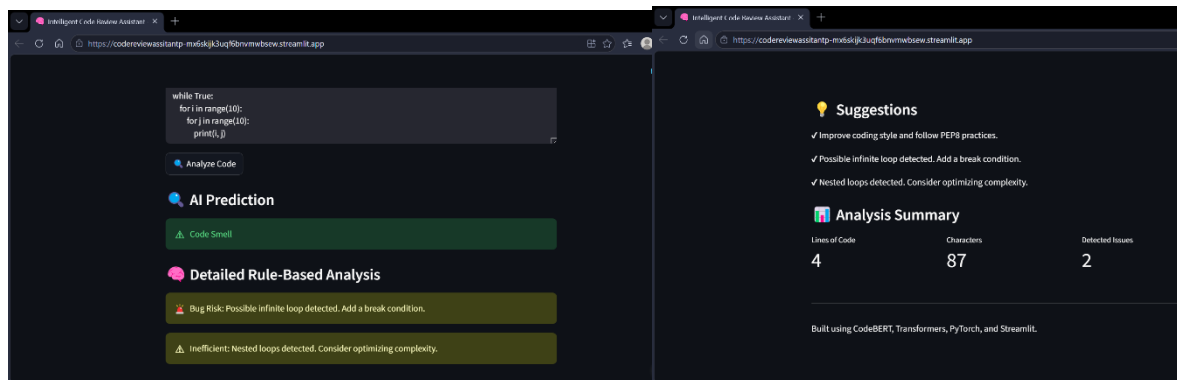


Figure 5: Automated Review Results and Suggestions

9.4 Deployment Platforms

The system can be deployed using:

- Streamlit Community Cloud
- Render
- Railway
- Google Colab with ngrok

9.5 Deployment Challenges

Challenges encountered:

- Large model loading
- Cloud memory limits
- Incorrect file paths
- Dependency installation issues

9.6 Deployment Solutions

Solutions implemented:

- Streamlit caching
- Lightweight deployment configuration
- Correct project structure
- requirements.txt optimization

Chapter 10 – System Features

10.1 AI-Based Prediction

The system predicts:

- Clean code
- Inefficient code
- Bug risks
- Code smells

10.2 Rule-Based Analysis

Additional rule-based checks improve reliability.

Detected issues:

- Infinite loops
- Nested loops
- Bad None comparisons
- Excessive print statements
- Long functions

10.3 Automated Suggestions

The system provides actionable recommendations.

Examples:

- Reduce algorithm complexity
- Replace `== None` with `is None`
- Add break conditions

10.4 Explainability

The system explains predictions using token importance scores.

Chapter 11 – Results and Discussion

11.1 System Performance

The evaluation phase yielded an F1-score of 0.8880, indicating a robust balance between precision and recall. The accuracy of 88.93% suggests that the CodeBERT-based model successfully generalized well to the unseen test data.

The close alignment of Precision (88.90%) and Recall (88.93%) implies the model is neither too conservative (missing bugs) nor too aggressive (flagging clean code). This balance is crucial for a code review assistant, as developers require reliable feedback without excessive noise.

11.2 Advantages

- Reduces manual review effort
- Detects inefficient patterns
- Provides practical suggestions
- Supports educational use

11.3 Limitations

- Labels generated heuristically
- Limited dataset size
- Supports only Python
- Not equivalent to full static analysis tools

11.4 Future Improvements

Future enhancements may include:

- Multi-language support
- Real labeled bug datasets
- Automatic code fixing
- Complexity analysis
- IDE integration
- Advanced explainability visualization

Chapter 12 – Conclusion

This project successfully designed and implemented an Intelligent Code Review Assistant using Deep Learning and NLP techniques.

The system combines transformer-based learning with rule-based analysis to detect bugs, inefficiencies, and code smells in Python programs.

The project demonstrates the complete machine learning lifecycle:

- Data collection
- Preprocessing
- Model training
- Evaluation
- Explainability
- Deployment

The final system provides real-time code review suggestions through a user-friendly web application and demonstrates the practical application of Artificial Intelligence in Software Engineering.

References

1. Feng, Z., Guo, D., Tang, D., et al. "CodeBERT: A Pre-Trained Model for Programming and Natural Languages."
2. Hugging Face Transformers Documentation.
3. Microsoft Research – CodeBERT.
4. CodeSearchNet Dataset Documentation.
5. Streamlit Documentation.
6. Captum Explainability Documentation.
7. PyTorch Documentation.
8. Scikit-learn Documentation.

Appendix

Technologies Used

Technology	Purpose
Python	Programming Language
PyTorch	Deep Learning Framework
Transformers	NLP Models
CodeBERT	Source Code Understanding
Streamlit	Web Application
Captum	Explainability
Scikit-learn	Evaluation Metrics
Hugging Face Datasets	Dataset Loading

Hardware Requirements

- Intel Core i5 or higher
- Minimum 8GB RAM
- GPU recommended

Software Requirements

- Python 3.10+
- PyTorch
- Transformers
- Streamlit
- Captum
- Scikit-learn

Example Input

while True:

for i in range(10):

for j in range(10):

print(i, j)

Example Output

Prediction:

- Bug Risk
- Inefficient Code

Suggestions:

- Add break condition
- Reduce nested loops
- Improve algorithm efficiency